LEAP: Hardware-Aware Learned Eviction for Line-Rate Persistent Flow Measurement on FPGAs and Programmable Switches

Technical Report (Supplementary Material)

Anonymous Authors

This report gives the proofs, details, and results that the paper "LEAP: Hardware-Aware Learned Eviction for Line-Rate Persistent Flow Measurement on FPGAs and Programmable Switches" points to. Unless stated otherwise, the setup is that of Section V of the paper: 2026 MAWI traces [1], $M = 1500$ windows, $\alpha = 0.4$, $d = 2$ rows, and five hash seeds.

S1. Theoretical Analysis

This section proves the properties of LEAP that the paper relies on: a flow that owns a bucket is never overestimated (Section S1-B); a new flow behind a stale owner waits a time that grows quadratically with the owner's count under the Pontus rule [2], and, early in the measurement period, much less under the learned table (Section S1-C); the two shortcuts of the FPGA pipeline are safe: the lazy roll-over matches the per-window sweep except after 16 or more idle windows, and the early threshold comparison never changes the result (Section S1-D); and the error of a flow that owns a bucket is the number of windows in which it appeared before it last took the bucket plus the number of its decays in windows in which it is absent, whose distribution we bound and compare with simulation (Section S1-E). Section S1-A recalls the update rule on which the proofs rest.

A. Setting

A bucket stores the fingerprint of its owner, the owner's count P, and three pieces of window state: the arrival flag F ($F = 1$ if the owner has arrived in the current window), the decay flag R ($R = 1$ if the bucket was decayed in the current window), and the presence bits $H = H_2H_1$ ($H_k = 1$ if the owner arrived in window $t - k$). A packet of flow e scans its d buckets in row order and stops at the first one that is empty or owned by e (Algorithm S1 in Section S2).

(U1) If this bucket is empty, e takes it with count 1.

(U2) If e owns this bucket, the first packet of e in a window raises the count by one and sets F; if the bucket was decayed earlier in the same window ($R = 1$), it raises the count by two instead, since the arrival shows that the decay hit a flow that is still present. Later packets of e in the same window find $F = 1$ and change nothing.

(U3) Otherwise, all d buckets are owned by other flows (a *full collision*). The owner m with the smallest count is decayed with probability π read from the table T, but only if $F = R = 0$, i.e., only if m has neither arrived nor been decayed in this window. A decay lowers the count by one and sets R; if the count reaches 0, e takes the bucket with count 1.

At the end of each window, every bucket shifts F into H and clears F and R. Three consequences are used below: a bucket is decayed at most once per window, never after its owner has arrived in that window, and, once taken, never becomes empty again, since a decay to count 0 hands it to e.

B. One-Sided Error

The count of an owner grows at most once per window in which the owner appears, and decays only lower it, so the count never exceeds the true number of windows.

Lemma S1 (one-sided error)**.** *If no other flow has the fingerprint of f, a flow f that owns a bucket at the end of window t satisfies $\hat{P}(f) \le P(f)$, for every policy π.*

Proof. Let f take its bucket in window t_0; its count is then 1, and f has appeared in t_0. Consider any later window. Only packets of f raise the count, because no other flow has its fingerprint. If f does not appear in the window, the count therefore cannot grow; it drops by one if the bucket is decayed (at most once, by (U3)). If f appears, its first packet raises the count by one, or by two if a decay removed one unit earlier in the window, so the net change of the window is $+1$ in both cases; later packets add nothing, and no decay can follow, since it requires $F = 0$. The count thus grows by at most one per window in which f appears, so at the end of window t it is at most the number of windows in $[t_0, t]$ in which f appeared, which is at most $P(f)$. □

The fingerprint assumption is needed because a flow with the same fingerprint would be taken for the owner by (U2) and could raise the count; for bucket owners, such collisions are the only source of overestimation. (A flow without a bucket is answered with the smallest count among its buckets, which can exceed its persistence.) The bound also holds with the lazy roll-over of the FPGA (Section S1-D), where a stale $R = 1$ can survive a stamp wrap-around: every transition preserves $c \le A - [R{=}1 \wedge F{=}0]$, where c is the count and A the number of windows since t_0 in which f appeared.

C. Entry Delay

A new flow can take a bucket only after the current owner's count has been decayed to zero, one unit at a time. Under the Pontus rule, a decay at count v succeeds with probability $1/(v+1)$, so a large count blocks the bucket for a long time even when its owner has stopped appearing. We consider a bucket owned by such a *stale* owner, which no longer appears, with count V. In each window, independently with probability ρ, exactly one packet meets this bucket as the minimum-count bucket of a full collision, and this packet belongs to the new flow; we call such a window a *collision window*. Let D be the number of windows, counted from the first window after the owner's last arrival, until the owner's count reaches 0, i.e., until the new flow takes the bucket. A *count-only* policy decays with a probability $\pi(v) > 0$ that depends only on the count; a *presence-aware* policy depends on the count and on H but not on time, and $\pi_{00}(v) > 0$ denotes its value for an owner seen in neither of the two previous windows. (The learned table also depends on time; it is treated below.)

Theorem S1 (entry delay)**.** *Under a count-only policy π,*

$$\mathbb{E}[D] = \sum_{v=1}^{V} \frac{1}{\rho\,\pi(v)}, \tag{S1}$$

which is $V(V+3)/(2\rho) = \Theta(V^2/\rho)$ for the Pontus rule $\pi(v) = 1/(v+1)$. Under a presence-aware policy, $\mathbb{E}[D] \le 2 + \sum_{v=1}^{V} 1/(\rho\,\pi_{00}(v))$.

Proof. The stale owner never arrives, so its flag F stays 0, and R is cleared at every window boundary. Each collision window therefore gives exactly one decay attempt, which is allowed because $F = 0$ and R was cleared at the window boundary. At count v, the attempt succeeds with probability $\pi(v)$, so each window lowers the count by one with probability $\rho\,\pi(v)$, independently of the other windows. The number of windows that the owner spends at count v is thus geometric with mean $1/(\rho\,\pi(v))$, and summing these means over $v = V, \dots, 1$ gives Eq. (S1). For the Pontus rule, $\sum_{v=1}^{V}(v+1) = V(V+1)/2 + V = V(V+3)/2$. For a presence-aware policy, the owner's presence bits are $H = 00$ from the third window after its last arrival; attempts in the first two windows can only lower the count further, and from then on each attempt succeeds with probability $\pi_{00}(v)$, so the same argument gives the bound. □

If a collision window brings k colliding packets instead, the first successful draw sets $R = 1$ and stops the others, so $\pi(v)$ becomes $1 - (1-\pi(v))^k$ in Eq. (S1) and the wait shortens. For example, behind a stale owner of count $V = 10$ with one collision in every window ($\rho = 1$), the Pontus rule needs 65 windows on average to free the bucket. During this wait, a persistent new flow appears in many windows that it cannot count, which is the late-insertion error of Fig. 2(a) of the paper.

Exact chains of the deployed table. The learned table also depends on the time bin, so Theorem S1 does not give its delay directly. We therefore compute the exact, time-inhomogeneous Markov chain of the stale owner's count under the same assumptions ($\rho = 1$, one attempt per window), with the entries of T for $H = 00$ in the time bin of each window, starting once the owner's presence bits are $H = 00$. For a wait that starts at the beginning of the period (time bin 0), the new flow behind an owner of count 10 gets its bucket after 20 windows on average with the learned table, against 65 with the Pontus rule. The table decays stale owners less often later in the period, so the same wait grows to 27, 39, and 58 windows when it starts after a quarter, a half, and three quarters of the period, and to 72, above the Pontus rule's 65, when it starts after seven eighths. Behind an owner of count 50, the learned table needs 320 windows from the start of the period; under the Pontus rule, the new flow gets the bucket within 1,500 windows only with probability 0.80 (1,325 windows on average by Theorem S1).

D. Correctness of the FPGA Shortcuts

Compared with Algorithm S1 executed packet by packet, the FPGA pipeline of Section IV-A of the paper changes two things. (1) It does not roll every bucket over at the end of a window, but applies the missed roll-overs when a packet next reads the bucket, using a 4-bit window stamp. (2) It performs the threshold comparison one stage early, in S4, where the update of the packet just ahead may not be visible yet. Lemma S2 shows that (1) gives the state of the per-window sweep except in rare cases; Theorem S2, that (2) never changes it.

A roll-over by one window maps (F, R, H) to $(0, 0, ((H \ll 1) \,|\, F)\,\&\,3)$: the arrival flag becomes the newest presence bit, the older presence bit drops out, and both flags are cleared.

Lemma S2 (lazy roll-over)**.** *(a) For $\delta \ge 1$, δ consecutive roll-overs map (F, R, H) to $\big(0, 0, ((H \ll \delta) \,|\, (F \ll (\delta - 1)))\,\&\,3\big)$. (b) With a 4-bit stamp, the pipeline rolls a bucket that was last accessed δ windows ago over by $\delta' = \delta \bmod 16$ windows (δ' is the δ of the paper). Its state differs from that of the per-window sweep only if $\delta \ge 16$ and $\delta' \in \{0, 1, 2\}$.*

Proof. (a) By induction on δ. The case $\delta = 1$ is the definition. If the claim holds for δ, one more roll-over clears F and R and shifts H left by one, which gives $((((H \ll \delta) \,|\, (F \ll (\delta - 1)))\,\&\,3) \ll 1)\,\&\,3 = ((H \ll (\delta + 1)) \,|\, (F \ll \delta))\,\&\,3$. (b) The stamp stores the window of the last access modulo 16, so the pipeline sees δ' missed windows. For $\delta \ge 3$, (a) gives $(0, 0, 00)$: the arrival flag and both presence bits have been shifted out, so any number of roll-overs of at least three gives the same state. (For $\delta = 2$, (a) gives $H = F0$: an arrival two windows ago is still the older presence bit.) Hence the pipeline is exact if $\delta < 16$, and also if $\delta \ge 16$ and $\delta' \ge 3$; only $\delta' \in \{0, 1, 2\}$ can leave stale bits after $\ge$16 idle windows. □

When packet i is in S4, the packet that entered one cycle earlier, $i-1$, is in S5, packet $i-2$ is in S6, and packets $i-3$ and $i-4$ have just been written to block RAM; the pipeline forwards the words of packets $i-2$ through $i-4$ to S4 and the word of $i-1$ to S5 (Fig. 5(a) of the paper).

Theorem S2 (early threshold comparison)**.** *Let the input of the pipeline leave a one-cycle gap whenever the window changes, and let the controller write the thresholds only while the pipeline is empty. Then, for every packet, the pipeline*

makes the same decision and writes the same bucket words as Algorithm S1 with the lazy 4-bit roll-over, applied to the packets in order with the same hashes and random numbers, starting from empty buckets, where each packet, as in the hardware, rolls over and writes back every bucket it reads. With the per-window sweep instead, the same holds up to the first read of a bucket after 16 or more idle windows (Lemma S2(b)); after it, the two runs may diverge.

Proof. We compare the pipeline with the sequential execution by induction on the packet index i: assume that all packets before i made the sequential decisions and wrote the sequential words. (i) *Same inputs.* The linear-feedback shift register (LFSR) advances once per valid input, so packet i uses the same random number as in the sequential execution, and the thresholds do not change while it is in flight. (ii) *One change per packet.* The rows are separate memories, so a bucket is a (row, index) pair and two rows never share one. A packet changes at most one of its buckets (the empty, matching, or minimum-count one) and only rolls the others over; so packet $i-1$ changed at most one of the buckets of packet i. (iii) *What S4 sees.* In S4, packet i uses the newest word of each of its buckets, rolled over to the current window: this word includes the writes of all packets up to $i-2$, and the only write it can miss is that of $i-1$, which is in S5 in the same cycle. If packet $i-1$ did not change the bucket, this stored word is the one the sequential execution reads, and S4 rolls it over to the current window exactly as the sequential execution does, so the comparison result is the sequential one. (iv) *What S5 does.* S5 evaluates the whole control flow of Algorithm S1 on the newest words, including the one of packet $i-1$: empty and matching buckets, the row order, the minimum count with ties to row 1, and the eligibility $F = R = 0$; only the result of comparing the random number with the threshold comes from S4. If packet $i-1$ changed bucket b of packet i in the same window, it inserted, counted, decayed, or replaced the owner of b, and each of these sets F or R. Bucket b then either matches packet i or is not eligible for a decay, both in the sequential execution and in S5, so the possibly stale comparison result for b is never used. By (ii), the comparison for the other bucket was made on its current state. (v) *Window boundaries.* The roll-over clears F and R, so the argument of (iv) does not hold if packets $i-1$ and i fall into different windows. The one-cycle input gap at the window change places packet $i-1$ two positions ahead of packet i, so its write reaches S4 by forwarding, and case (iii) applies. In all cases, packet i makes the sequential decision and writes the sequential words. □

E. Error Bound of a Tracked Flow

As HeavyKeeper [3] does for the flows it tracks, we bound the error of a flow that holds a bucket. Fix a flow f, the set S of windows in which it appears, and a window t_0 in which f takes a bucket B of row r. For a later window w, let $A_w = |S \cap [t_0, w)|$ be the number of windows from t_0 on in which f appeared before w, and let H_w be the presence bits of B in w, i.e., the presence of f in windows $w-2$ and $w-1$, counted as absent before t_0 (a new owner starts with $H = 00$). Let $\pi(\kappa, H, P)$ be the decay probability of the policy: $T[\kappa][H][b(P)]/2^{16}$ for the learned table and $1/(P+1)$ for the Pontus rule. For a bucket B' and a packet, let $N_{B'}$ be the largest number, over all fingerprints ϕ, of windows up to and including that packet in which some flow that hashes to B' and has fingerprint ϕ appeared. For $c \geq 1$, let $K_w(c)$ be the number of packets in window w of flows $g \neq f$ with $h_r(g) = B$ such that, at that packet, $N_{B'} \geq c$ for every bucket B' of g in the other rows. $K_w(c)$ depends only on the trace and the hash functions, and it does not increase with c. We assume that (A1) no flow other than f that hashes to one of the buckets of f has the fingerprint of f; (A2) each decay attempt draws a fresh random number, independent of everything before it; and (A3) π does not increase with the count, which holds for the Pontus rule and for each of the 32 (time bin, H) rows of the deployed table.

Theorem S3 (error of a tracked flow)**.** *Under (A1)–(A3), fix the trace, the hash functions, and the history up to the end of window t_0. For $t > t_0$, let $W = (t_0, t] \setminus S$ and let D_t be the number of windows $w \in W$ in which B is decayed while f has held B without interruption since t_0 (decays after f first loses B do not count). (a) If f owns B throughout $(t_0, t]$, then $P(f) - \hat{P}(f) = |S \cap [1, t_0)| + D_t$. (b) For every $k \geq 0$, $\Pr[D_t > k] \leq \beta_k := \Pr[\sum_{w \in W} Y_w > k]$, where the Y_w are independent Bernoulli variables with means*

$$q_w(k) = 1 - \big(1 - \pi(\kappa(w), H_w, c_w)\big)^{K_w(c_w)},$$

with $c_w = \max(1, A_w - k)$. (c) Hence $\Pr[D_t > k] \leq \frac{1}{k+1} \sum_{w \in W} q_w(k)$ and $\mathbb{E}[D_t] \leq \sum_{k=0}^{|W|-1} \min_{j \leq k} \beta_j$.

Proof. (a) By (A1), only packets of f raise the count of B, and every packet of f reaches B: the buckets of f in earlier rows were not empty when f took B, buckets never become empty again, and by (A1) their owners do not have the fingerprint of f. f took B with count 1 in $t_0 \in S$. As in Lemma S1, each window of $(t_0, t]$ in which f appears changes the count by exactly $+1$: its first packet adds one, or two if B was decayed earlier in the window, and no decay can follow it. Each other window lowers the count by one if B is decayed in it, which happens at most once, and leaves it unchanged otherwise. Hence $\hat{P}(f) = |S \cap [t_0, t]| - D_t$, whereas $P(f) = |S \cap [1, t]|$.

(b) The same accounting shows that, as long as f has held B without interruption since t_0, the count of B at the start of window w is $V_w = A_w - D_{w-1}$. Let $w \in W$, and assume that f has held B without interruption from t_0 to the start of w and that $D_{w-1} \leq k$, so that $V_w \geq c_w$. No packet of f arrives in w, so the count, H_w, and $\kappa(w)$ stay fixed until B is decayed, and the first decay sets R and rules out further decays in w. A packet of a flow $g \neq f$ can decay B only if $h_r(g) = B$ and B has the smallest count among the buckets of g (ties go to the earlier row), so every other bucket B' of g holds a count of at least V_w. By the argument of Lemma S1, a count grows by at most one per window in which a flow that hashes to the bucket and has its stored fingerprint appears, and it restarts at 1 when the bucket changes owner; so the count of B' is at

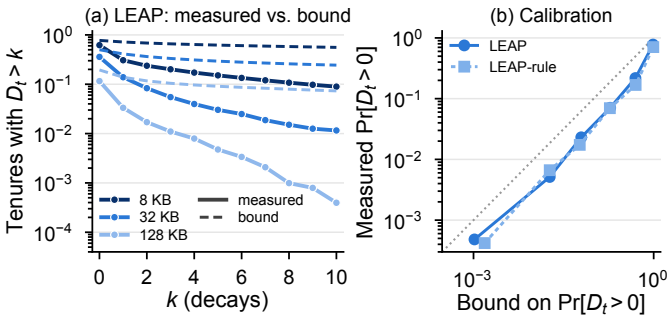


Fig. S1. Theorem S3 against simulation (all tenures of persistent flows, including those that end in an eviction; four MAWI test traces, three seeds). (a) Fraction of tenures with more than k decays: measured (solid) and the average bound of part (b) (dashed). (b) Tenures grouped by their bound on $\Pr[D_t > 0]$: measured frequency per group; points below the diagonal satisfy the bound.

most $N_{B'}$, and $N_{B'} \geq V_w \geq c_w$ at that packet. Hence only the packets counted by $K_w(c_w)$ can try to decay B. Order them: given everything before the j-th, it decays B with probability at most $\pi_w = \pi(\kappa(w), H_w, c_w)$ if it tries, by (A2), and by (A3) since $V_w \geq c_w$, and with probability 0 otherwise; by induction on j, none of them decays B with probability at least $(1-\pi_w)^{K_w(c_w)}$. So the conditional probability p_w that B is decayed in w while f still holds it, given the history before w ($p_w = 0$ if f has lost B since t_0), is at most $q_w(k)$ whenever $D_{w-1} \leq k$. To compare D_t with $\sum_w Y_w$, generate the run window by window: draw an independent uniform U_w, set the decay indicator to $X_w = \mathbf{1}\{U_w < p_w\}$, and draw the rest of window w from its conditional distribution given the history and X_w; this leaves the joint distribution unchanged. Let $Y_w = \mathbf{1}\{U_w < q_w(k)\}$. As long as at most k decays have occurred, $p_w \leq q_w(k)$ and thus $X_w \leq Y_w$; so if $D_t > k$, the Y_w of the windows up to the $(k{+}1)$-th decay already sum to at least $k+1$.

(c) The first bound is Markov's inequality for $\sum_w Y_w$. Since $\Pr[D_t > k]$ does not increase with k, it is at most $\min_{j \leq k} \beta_j$, and $\mathbb{E}[D_t] = \sum_{k=0}^{|W|-1} \Pr[D_t > k]$ because $D_t \leq |W|$. □

The error of a tracked flow thus has two parts: the windows in which it appeared before it last took its bucket, which Theorem S1 and the chains of Section S1-C bound behind a stale owner, and its decays in windows in which it is absent, which the theorem bounds from the trace, the hash functions, and the policy alone. Three effects keep these decays rare for a persistent owner: its count, and with it its protection, grows with A_w; only packets whose other buckets can hold an equally large count can decay it; and the learned table decays an owner seen in the previous window ($H = 01$ or 11) with probability at most 0.041 at count 2 (Pontus: 1/3) and below $2 \cdot 10^{-4}$ at count 100 (Pontus: 0.0099).

Comparison with simulation. We instrumented the simulator to follow every *tenure* of a persistent flow, from a window t_0 in which it takes a bucket until it is evicted or the period ends, and to record its decays in windows in which it is absent; a second pass with the same hash functions records its presence and the packets counted by $K_w(c)$. Tenures start at takeovers and include those that end in an eviction, so none is selected by its future and part (b) applies to each. Across LEAP and LEAP-rule at 8, 32, and 128 KB on the four MAWI test traces with three seeds each, there are 104,258 tenures; we exclude the 5,358 tenures (5.1%, mostly at 8 KB) of flows that violate (A1). Part (a) holds exactly for each of the 52,730 tenures that last until the end of the period. Fig. S1(a) compares the measured fraction of tenures with more than k decays with the average bound of part (b): the measurement stays below the bound at every k, for both policies and all three sizes (at most 0.80 of the bound), and for LEAP the bound exceeds it by 1.2–1.7× at $k = 0$ (e.g., 0.50 vs. 0.36 at 32 KB). Grouping the tenures by their bound on $\Pr[D_t > 0]$ (Fig. S1(b)) gives the same in every group, from $5 \cdot 10^{-4}$ vs. 10^{-3} to 0.78 vs. 0.98. The bound is conservative mainly because $K_w(c)$ counts every packet that could find B at the minimum, not only those that do. A separate, independently written simulator tested the theorem flow by flow: on small adversarial traces ($d = 2$ and 3, two to sixteen buckets per row, both policies) with fixed hash functions and 20,000 random seeds per case, no measured probability exceeded the bound, and at $k = 0$ the bound is attained when every packet counted by K_w is sure to find B as the eligible minimum. Decays are also the smaller part of the error of a tracked flow: at 32 KB, a LEAP owner at the end of the period has missed 2.65 windows before taking its bucket and lost 0.70 to decays on average.

Algorithm S1 LEAP update for an arriving flow e in window t (Section III-A and Fig. 4 of the paper)

```
1: g ← fp(e); m ← ⊥
2: for i = 1 to d do
3:     B ← bucket h_i(e) of row i
4:     if B is empty then B ← (g, P=1, F=1, R=0, H=00); return
5:     if B.fp = g then
6:         if B.F = 0 then B.P ← B.P + (B.R ? 2 : 1); B.F ← 1
7:         return
8:     if m = ⊥ or B.P < m.P then m ← B
9: if m.F=0 ∧ m.R=0 ∧ rnd_16 < T[κ(t)][m.H][b(m.P)] then
10:    m.P ← m.P − 1; m.R ← 1
11:    if m.P = 0 then m ← (g, P=1, F=1, R=0, H=00)
```

S2. Training and the Learned Policy

Update and table. Algorithm S1 is the update of the paper; it differs from Pontus only in line 9, where the lookup in T replaces the probability $1/(P+1)$. The policy model contains a close variant of this rule: setting only the weight on u to $-\ln(P_{\max}+1)$ gives $\pi_\theta = 1/(P+2)$. At the end of each window, every bucket shifts F into H and clears F and R (lazily on the FPGA, Lemma S2).

Two stages. Stage 1 runs the Pontus rule on the January trace and logs 13.8 M decay opportunities; a logistic classifier predicts whether the owner becomes persistent with an area under the receiver-operating-characteristic curve (AUC) of 0.96 (accuracy 97.1%, against 94.6% for always predicting "not persistent"). Stage 2 starts the covariance matrix adaptation evolution strategy (CMA-ES) [4] from the Stage-1 policy (population 8, step size 0.6) and scores each candidate on the January trace at 16, 32, and 64 KB with two hash seeds;

TABLE S1
MEAN AAE (WINDOWS) AFTER EACH TRAINING STAGE, OVER THE FOUR MAWI TEST TRACES WITH FIVE SEEDS EACH.

Memory (KB)	8	16	32	64	128
LEAP-rule	60.2	22.6	8.92	2.92	0.97
LEAP-Stage 1	56.2	22.4	8.33	3.13	1.09
LEAP (after Stage 2)	44.7	17.4	6.47	2.13	0.67

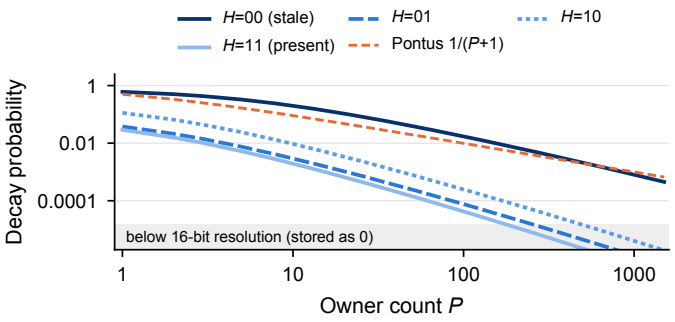


Fig. S2. Learned decay probability versus the owner's count for each presence history H (at mid-period, $\tau = 0.5$), compared with the Pontus rule.

after 14 generations, when its step size has shrunk, it restarts from the best weights with the initial step size for 12 more; the best candidate reaches 0.63 of the rule's average absolute error (AAE) on the training trace. The policy never sees the test traces. Two further searches with other CMA-ES seeds end at 0.64 and 0.66 of the rule's AAE on the training trace, and on the four test traces they lower the AAE of LEAP-rule by 20–25% and 11–30% across memory sizes (deployed table: 23–31%); every search beats the rule at every memory size, and the training scores rank the three searches as their average test results do.

What each stage contributes. Table S1 follows the training (Section V-C of the paper). Stage 1 alone changes the AAE of LEAP-rule by -7% to $+12\%$, as it fits persistence labels rather than the AAE; the search then lowers the AAE of the Stage-1 policy by a further 20–38%. The Stage-1 policy gives the search a better start where memory is scarce: a single 26-generation search started from the Pontus rule instead ends with an 18% higher AAE than LEAP at 8 KB and a 5% higher AAE on average over 8–128 KB; at 128 KB, the two starts end within 0.1 windows of each other.

What the policy learned. Fig. S2 shows a "use it or lose it" rule. At mid-period ($\tau = 0.5$, as in the figure), owners absent from both previous windows (H=00) are decayed 1.5–2.2× more often than under the Pontus rule at counts 2–100, so stale owners release their buckets sooner; later in the period, the table decays them less often (Section S1-C). Owners present in both windows (H=11) are decayed 21×, 47×, and 232× less often at counts 2, 10, and 100, and recent presence (H=01) is worth more than older presence (H=10), although H=10 owners end up persistent more often (1.4–1.9× on average over the seven traces; Fig. 2(b) of the paper): the search minimizes the AAE, not the error of a persistence label. Because protected owners are rarely decayed, LEAP also decays less often at 8–64 KB (on 1.8–7.6% of the packets versus 3.1–7.9% for

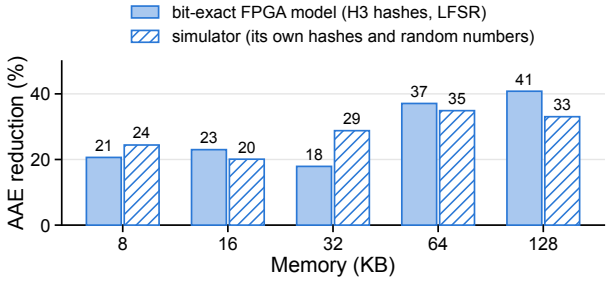


Fig. S3. AAE reduction of the learned table over the Pontus rule compiled into the same table, mean over the four MAWI test traces: bit-exact FPGA model (five draws of the H3 masks and the LFSR state) and the simulator (five seeds).

LEAP-rule), which lowers the recirculation load on Tofino.

S3. FPGA PIPELINE

Read-after-write hazards are resolved by forwarding from S6 (Fig. 5(a) of the paper): S4 uses the words of the packets two to four positions ahead (the one in S6 and two history registers behind it), and S5 the word of the immediately preceding packet; each forwarding select is computed one stage early (in S3 for S4, in S4 for S5).

Why the comparison is made in S4. To measure what the early threshold comparison saves, we built a variant of the core that looks up the threshold and compares the random number in S5, on the word forwarded from the packet just ahead. It computes the same results (it passes the 400 K-packet golden test below bit for bit), but its critical path chains the forwarding multiplexer, the count-bin logic, the threshold read, the 16-bit comparison, and the update logic in one cycle. At 32 KB on the ECP5-85F, it reaches 38.6–42.2 MHz over ten placement seeds (median 40.3), versus 66.0–69.0 MHz (median 67.3) for the design.

Lazy roll-over. A bucket that is not accessed for 16 or more windows sees its 4-bit stamp wrap. On the four test traces at 8–128 KB, 0.055% of the 1.2 G bucket reads follow 16 or more idle windows (mostly at 128 KB), and replacing the lazy roll-over by an exact one changes the AAE by at most 0.7%.

Verification. In Icarus Verilog, the per-packet estimates of the register-transfer-level (RTL) design match a bit-exact C++ model on 400 K April MAWI packets with the rule table, on 1 M July MAWI packets with random tables and input gaps, and on a stress trace with a window boundary every three packets; without the gap, 12.5% of its estimates differ. The gaps at window changes and the threshold reloads at the seven time-bin changes (a pipeline drain and 96 writes each) take under 0.01% of the input cycles of a period.

The learned table on the hardware. The bit-exact model of the RTL uses the H3 hashes, the 16-bit LFSR, and the 4-bit stamps of the FPGA instead of the simulator's hash function and random numbers. Fig. S3 shows that the learned table gains 18–41% over the compiled rule on it, close to the 20–35% of the simulator; the largest gap is at 32 KB (18% versus 29%).

TABLE S2

PACKET-LEVEL MODEL OF THE COMPILED TOFINO PROGRAM VERSUS THE SIMULATOR AT 32 KB (AAE; MODEL: TEN SEEDS; SIMULATOR: FIVE SEEDS AND THEIR RANGE).

	April	July	October
Packet-level model, immediate second pass	6.13	3.15	8.52
Simulator, mean	6.61	3.51	8.37
range over seeds	3.71–11.33	2.39–4.76	7.22–10.57
Packets that recirculate (model)	5.4%	6.9%	4.7%

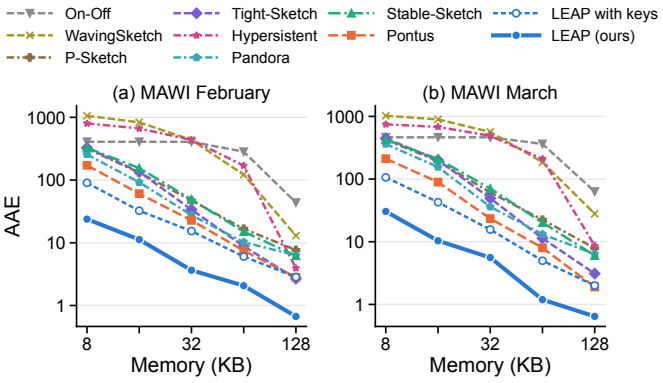


Fig. S4. AAE on the two held-out MAWI traces of February and March 2026 (mean of five seeds).

S4. TOFINO PROGRAM

Each bucket keeps its fingerprint, count, w_a, w_d, and p in five register arrays (96 bits), since stateful arithmetic logic units (ALUs) cannot pack them; the control plane also installs the 22 ternary count-bin entries per row and clears the registers at the start of each measurement period. The second pass compares and swaps only the fingerprint, so an owner of count 1 that arrives while the decay request recirculates is still replaced; our model, with an immediate second pass, does not exercise this race. Our packet-level model executes the compiled program's register actions in pipeline order; its AAE lies within the simulator's range over hash seeds on all three traces (Table S2). On Intel's Tofino model, a reduced build (256 buckets per row, random numbers taken from the packet so that the run is deterministic) reproduces all 2,560 register words of our C++ reference after 6,000 and 40,000 MAWI packets, and its AAE after 40,000 packets (1.98) lies within the simulator's range over 30 seeds (1.06–2.26).

S5. ADDITIONAL RESULTS

More backbone traces. On February and March 2026 (Fig. S4), LEAP is 3.0–8.6× more accurate than the best baseline at every memory size.

Other datasets and window counts. CTU-13 [5] and StackOverflow [6] are split, like the MAWI traces, into $M = 1500$ windows with equal numbers of items, taken in time order: one item per CTU-13 flow record, keyed by its IPv4 source–destination pair, and one per StackOverflow interaction, keyed by the pair of users. At $M = 3000$ (Fig. 8(f) of the paper), counts need 12 bits, so LEAP and LEAP-rule use 16-bit fingerprints there.

TABLE S3

COUNT BINNING: AAE CHANGE (%) AGAINST THE DEPLOYED TABLE, MEAN OVER THREE TEST TRACES × THREE SEEDS (PER-TRACE RANGE).

Count bins	Size	8 KB	32 KB
1 per doubling	0.7 KB	−5.1 (−8, +1)	+10.7 (−4, +20)
2 per doubling (deployed)	1.5 KB	0	0
4 per doubling	2.8 KB	+0.9 (−4, +8)	+5.5 (−5, +17)
24 linear (64 counts each)	1.5 KB	+20.1 (+18, +24)	+63.4 (+47, +70)

Count binning of the table. Table S3 compiles the trained policy with other count binnings (Section III-A of the paper), each bin taking the policy's value at the middle of its count range, and evaluates the tables (8 time bins, 16-bit thresholds) on the April, July, and October traces as in Fig. 8(c, d) of the paper. How finely the logarithmic scale is divided matters little: one bin per doubling halves the table, and four bins per doubling do not improve the AAE. The logarithmic shape matters: 24 linear bins of 64 counts, a table of the same size as the deployed one, raise the AAE on each trace, by 20% on average at 8 KB and 63% at 32 KB, because one threshold then covers every count from 1 to 63, over which the Pontus probability falls from 1/2 to 1/64.

Fingerprint bits for presence. Against a version with the same number of buckets whose fingerprint keeps the two presence bits, taking them from the fingerprint changes the AAE by −9.9% to +3.3% across memory sizes and by −1.2% on average over paired runs on the four test traces.

Reporting persistent flows. At 32 KB, the FPGA core with the threshold comparator uses 2,443 LUT4s, 92 more than without it (median clock 65.4 MHz over ten placement seeds), and the Tofino program with the digest fits into the same 11 stages. On the MAWI test traces and CTU-13 [5], no report was a false positive, a flow was reported twice only after it was decayed below the threshold and crossed it again, and the controller never received more reports than there are persistent flows (at most 944 reports per trace).

Baseline settings. P-Sketch and Tight-Sketch use 3 rows and Pandora 4, as in their implementations; WavingSketch spends half of its memory on a Bloom filter that deduplicates arrivals per window. Hypersistent Sketch [7] was designed for windows of 1,500 packets; with 20 K packets per window, its filter counters saturate, so we tuned its memory split on the January trace and use an 80% filter share with threshold 255 (its artifact: 60%).

S6. ARTIFACT

The anonymous repository [8] contains LEAP for CPUs, the FPGA RTL with its bit-exact model and test benches, and the P4 program for Tofino, together with the learned table and the code that trains it.

REFERENCES

[1] MAWI Working Group, "MAWI Working Group Traffic Archive, samplepoint-F," https://mawi.wide.ad.jp/mawi/, 2026.

[2] W. Li, Z. Li, B. Bütün, A. F. Diallo, M. Fiore, and P. Patras, "Pontus: A memory-efficient and high-accuracy approach for persistence-based item lookup in high-velocity data streams," in *Proc. ACM WWW*, 2025, pp. 1783–1794.

[3] J. Gong, T. Yang, H. Zhang, H. Li, S. Uhlig, S. Chen, L. Uden, and X. Li, "HeavyKeeper: An accurate algorithm for finding top-k elephant flows," in *Proc. USENIX ATC*, 2018, pp. 909–921.
[4] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evol. Comput.*, vol. 9, no. 2, pp. 159–195, 2001.
[5] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Comput. Secur.*, vol. 45, pp. 100–123, 2014.
[6] A. Paranjape, A. R. Benson, and J. Leskovec, "Motifs in temporal networks," in *Proc. ACM WSDM*, 2017, pp. 601–610.
[7] L. Cao, Q. Shi, W. Xiao, N. Wang, W. Li, Z. Li, W. Zhang, and M. Xu, "Hypersistent Sketch: Enhanced persistence estimation via fast item separation," in *Proc. IEEE ICDE*, 2025, pp. 3030–3042.
[8] Anonymous, "LEAP: Technical report and code," https://anonymous.4open.science/r/Leap-DAC/, 2026.